Assignment 1

Process Abstractions, Scheduling & Performance Analysis

1. Process Abstraction & the POSIX API

1.1 State Machines Under Pressure

Modern kernels define **five** process states: *New*, *Ready*, *Running*, *Blocked*, and *Terminated*.

(a) A process is executing a tight arithmetic loop when the hardware timer fires. Trace the **exact sequence of kernel actions** that occur between the timer interrupt and the moment a different process resumes execution on the CPU. Your answer must name the specific data structures the kernel reads and writes (e.g. PCB, run queue, context).

(b) A student attempting to describe the lifecycle of a process that calls `read()` on a network socket with no data available claims the following sequence of transitions occurs:

$$\textit{Running} \rightarrow \textit{Terminated} \rightarrow \textit{Blocked} \rightarrow \textit{Ready} \rightarrow \textit{Running}$$

i. Identify **every** invalid transition in this sequence. For each one, state precisely why it is impossible according to the kernel's state machine rules.

ii. Write the **correct** transition sequence for this scenario, from the moment `read()` is invoked until the process resumes execution after data arrives. For each transition in your corrected sequence, name the triggering event and identify whether it originates from the process itself, the OS scheduler, or a hardware interrupt.

(c) A zombie process is one that has terminated but whose exit status has not yet been collected by its parent. Explain **why a zombie still occupies an entry in the process table** even though it is no longer executing. What specific system call removes the zombie, and what happens if the parent process exits before collecting it?

1.2 Deep `fork()` Analysis

Analyse the following C program carefully.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int g = 10;

int main() {
    int x = 5;
    int rc1 = fork();           /* Fork A */

    if (rc1 == 0) {
        int rc2 = fork();       /* Fork B */
        if (rc2 == 0) {
            g += 100;
            x += 100;
            printf("C: g=%d x=%d\n", g, x);
        } else {
            wait(NULL);
```

```
            printf("B: g=%d x=%d\n", g, x);
        }
        exit(0);
    }

    wait(NULL);
    printf("A: g=%d x=%d\n", g, x);
    return 0;
}
```

(a) **Process proliferation.** If `fork()` were instead called inside a loop that executes n times (with no `exit()` inside the loop), derive a general formula for the total number of processes that would exist after the loop completes. Explain the reasoning behind your formula. Then verify your formula against the actual number of processes created by the program above, being precise about why the structure of this program does *not* match the unconstrained loop case.

(b) **Exact output.** State the *complete and exact* output that will be printed. For each line, state which process prints it and justify the numerical values of `g` and `x`, explaining why the global variable `g` does *not* propagate changes between processes.

(c) **Ordering guarantees.** Process B calls `wait(NULL)` before printing, and Process A calls `wait(NULL)` before printing. Given these constraints, list *all* output orderings that are **impossible**, and prove why each is impossible using the happens-before relationship.

(d) **Modified scenario.** Suppose both `wait(NULL)` calls are removed and `x` is replaced by a pointer to heap-allocated memory (`int *x = malloc(sizeof(int)); *x = 5;`).

i. Would the child and grandchild now share the same heap object? Justify your answer with reference to the OS mechanism involved.

ii. What class of bug can arise when the parent frees the heap memory before the child has finished using it, and how would you detect it?

iii. Suppose Process B calls `exit(0)` without first calling `wait()` for Process C, which is still running. What is the state of Process C immediately after B exits? Which OS mechanism is responsible for handling this situation, and what becomes Process C's new parent? What broader system resource risk does this create if it happens repeatedly?

2. Scheduling Algorithms & Trade-offs

2.1 Multi-Process Workload Analysis

Consider the following workload. Processes arrive at the times shown and have the burst times given.

Process	Arrival Time (ms)	CPU Burst (ms)	Priority (lower = higher)
P1	0	10	3
P2	2	4	1
P3	4	6	2
P4	6	2	4
P5	8	8	2

(a) **Reasoning about preemption.** Consider the SRTF schedule for this workload. Without simulating the full schedule, answer the following:

i. Identify **every tick** at which a preemption event must occur. For each, name the process being preempted, the process taking over, and explain precisely why that tick is a decision point — i.e. what changes at that moment that forces the scheduler to act.

ii. P1 has the longest burst time and arrives first. Explain whether P1 can ever be the *last* process to finish under SRTF, and under what structural condition on the other processes' arrivals and bursts this would be guaranteed or impossible.

(b) **Algorithm comparison without simulation.** Using only the structure of the workload — arrival times, burst sizes, and their relationships — argue *without computing any wait times* why non-preemptive SJF must produce a lower or equal average wait time than FCFS on this specific workload. Identify the precise process or processes responsible for the difference and explain the mechanism.

(c) For **Round Robin** with $q = 3$ ms, compute the **average wait time** and **average turnaround time**, showing individual values for every process. Then explain why Round Robin's average turnaround time is higher than SRTF's for this workload, even though Round Robin is designed to be "fair."

(d) **Non-preemptive SJF** (ties broken by arrival order): compute the **average wait time** and **average turnaround time**, showing individual values for every process.

2.2 Theoretical Bounds

(a) **SJF optimality.** It is a well-known result that non-preemptive SJF minimises *average wait time* among all non-preemptive schedulers for a batch of jobs that all arrive simultaneously. **Prove this claim** for the case of $n = 3$ jobs with burst times $b_1 \leq b_2 \leq b_3$, all arriving at $T = 0$. Your proof must be general (in terms of b_1, b_2, b_3) and must show that any other ordering produces a higher or equal average wait time.

(b) **The SRTF–RR trade-off.** For a workload containing one long CPU-bound job (L ms) and k short interactive jobs (each s ms, $s \ll L$), with all jobs arriving simultaneously:

i. Derive a closed-form expression for the *average response time* under SRTF and under Round Robin with quantum q ($q < s$).

ii. Find the condition on k, s, and L under which RR produces a *lower* average response time than SRTF. Interpret this result intuitively.

3. Quantitative Performance & Advanced Metrics

Reference Definitions. The following metrics are used throughout Part 3. Let x_i denote the *wait time* of process i, let f_i denote its *finish time*, let a_i denote its *arrival time*, and let n be the total number of processes.

Wait Time. The total time a process spends in the ready queue before its first (or only) execution begins. Formally, $x_i = \text{start time}_i - a_i$ for non-preemptive schedulers. For preemptive schedulers, it is the total time spent waiting across all scheduling intervals: $x_i = (f_i - a_i) - b_i$, where b_i is the CPU burst time.

Turnaround Time. The total elapsed time from arrival to completion: $T_i = f_i - a_i$. It encompasses both waiting time and actual execution time, and represents the end-to-end latency experienced by the process.

Makespan. The wall-clock time to complete the entire batch of jobs, measured from

the earliest arrival to the last completion:

$$\text{Makespan} = \max_i(f_i) - \min_i(a_i)$$

Makespan captures the *throughput* of the scheduler — a lower makespan means more work done per unit time — but says nothing about how fairly that time was distributed across individual processes.

Jain's Fairness Index. A dimensionless measure of how equally wait time is distributed across all processes. If all processes wait the same amount, $J = 1.0$ (perfect fairness). If one process absorbs all the waiting while the rest wait zero, $J = \frac{1}{n}$ (worst-case starvation):

$$J = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}, \quad J \in \left[\tfrac{1}{n}, 1\right]$$

Note that J is sensitive to *variance*, not just the mean: two schedulers with the same mean wait time can have very different fairness indices if one produces a more uneven distribution.

P90 Wait Time. The 90th percentile of the wait time distribution: the value w such that 90% of all processes experienced a wait time $\leq w$. Unlike the mean, the P90 exposes the *tail behaviour* of a scheduler — a low mean with a high P90 indicates that most processes are served quickly but a minority suffer disproportionately long waits, which is characteristic of starvation-prone algorithms.

$$\text{P90 Wait} = \text{90th percentile of } \{x_1, x_2, \ldots, x_n\}$$

3.1 Metric Derivation

A scheduler produces the following execution trace on five processes, all arriving at $T = 0$:

Process	Start Time (ms)	Finish Time (ms)
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

(a) Calculate the **wait time** and **turnaround time** for each process.

(b) Calculate the **Makespan**.

(c) Calculate **Jain's Fairness Index** J using the wait times from (a). Show all arithmetic.

(d) This trace is consistent with FCFS on processes ordered P1 through P5. Now suppose the *same five burst times* are scheduled using **SJF**. Without performing the full execution trace, **predict** (with justification) whether J_{SJF} will be greater than, less than, or equal to J_{FCFS} you computed in (c).

3.2 Interpreting Real-World Measurements

A production team benchmarks four candidate schedulers on a workload of 10,000 processes. The results are summarised below.

Scheduler	Mean Wait (ms)	P90 Wait (ms)	P99 Wait (ms)	Makespan (ms)
Alpha	8	12	18	950
Beta	6	280	620	900
Gamma	11	14	19	870
Delta	9	11	16	1100

(a) **Identify the algorithm family.** Scheduler Beta has a mean wait of only 6 ms yet a P99 of 620 ms. Name the most likely scheduling algorithm family responsible for this pattern. Construct a concrete minimal example (you may use 3 processes) that reproduces the same qualitative pattern: low mean, extreme tail.

(b) **Throughput vs. tail latency.** Scheduler Gamma has the lowest Makespan (870 ms) but a higher mean wait than Alpha and Beta. Explain in one paragraph how a scheduler can simultaneously minimise Makespan while producing higher average wait times. Is Makespan a useful metric for interactive workloads? Justify your answer.

(c) **Deployment decision.** You are deploying *two* separate systems: (i) a batch genomics pipeline that runs overnight and cares only about total throughput; (ii) a real-time chat application where any message taking more than 50 ms to be processed causes a visible delay. For each system, select the best scheduler from the table and justify your choice using at least **two** metrics from the table.